MIT 8103 Advanced Database Systems: Individual Practical Database Portfolio

Name: Kolawole Abdulazeez || **Student ID: 301894623** || **Academic session:** 2026/2027
GitHub repository: https://github.com/Kolawoleabdulazeez/advanced-database-portfolio.git

1. Case Study

Shoptackles is a multi-vendor e-commerce platform where independent vendors list products and customers can browse, compare and purchase products from several vendors in a single order. It is a product I have worked on, but all data used in this portfolio is simulated. The same case study is used across all five portfolios.

- **Users:** Customers, vendors, administrators and delivery agents.
- **Major data:** Customers and addresses, vendors, categories, products, vendor offers (price and stock), orders and order items, payments, deliveries, delivery agents and reviews.
- **Key operations:** Product listing, multi-vendor ordering, stock updates, payment attempts, vendor-specific deliveries, product reviews and sales reporting.

2. Portfolio 1: Database Design and Modelling

I designed a PostgreSQL 17 database with 12 tables and created its ER diagram in dbdiagram.io (Figure 1).

Three junction tables resolve many-to-many relationships: vendor_products links vendors to products, order_items links orders to vendor offers, and reviews links customers to products.

Vendor-specific prices are stored in vendor_products, while order_items.unit_price preserves the purchase-time price. Orders can have multiple payment attempts, but a partial unique index allows only one successful payment. Each order can also have separate deliveries for each vendor.

The database follows 3NF, with unit_price and total_amount retained as deliberate historical and calculated values. Six constraint tests covering duplicate payments, negative quantities, duplicate emails, invalid ratings, missing customers and customer deletion with existing orders were rejected as expected (Figure 2).

3. Portfolio 2: Query Processing and Optimisation

I generated 50,003 customers, 200,003 orders and 400,005 order items. I then measured five business queries using EXPLAIN (ANALYZE, BUFFERS) before and after adding six indexes.

Query	Before	After	Improvement
Q1: Customer order history	139.6 ms	0.36 ms	About 390x faster
Q2: Vendor sales, last 90 days	124.4 ms	76.6 ms	About 1.6x faster
Q3: Price comparison for one product	5.8 ms	0.67 ms	About 8.6x faster
Q4: Best sellers, last 30 days	264.9 ms	175.8 ms	About 1.5x faster
Q5: Vendor shipments awaiting delivery	115.4 ms	3.4 ms	About 34x faster

Q1 and Q5 improved the most because full table scans accounted for much of their execution time. Q3 had a smaller improvement in absolute terms because its tables were relatively small.

Q2 and Q4 showed more limited improvements. Q2 still required about 2,000 lookups into the orders table, while Q4 continued to scan the order_items table because the query planner estimated that a sequential scan was cheaper than performing thousands of index lookups.

4. Portfolio 3: Transactions and Concurrency

Using two pgAdmin sessions, I demonstrated transactions with COMMIT and ROLLBACK. An order for 15 units against available stock of 10 failed a CHECK constraint, and the earlier inserts in the transaction were rolled back (Figure 3).

I also demonstrated a lost update where two sessions each read one available unit and processed a sale, resulting in two orders. I resolved this using SELECT ... FOR UPDATE, which made the second session wait until the first transaction completed (Figure 4).

I compared isolation levels, observing that READ COMMITTED returned an updated price while REPEATABLE READ retained the original. Finally, the partial unique index blocked a second successful payment with error 23505.

5. Portfolio 4: NoSQL and Advanced Data Models

I modelled Shoptackles in MongoDB using two main collections: products and orders. The products collection supports flexible attributes, such as phone RAM and storage or dress sizes, alongside embedded vendor offers. The orders collection stores customer details, items, payments and deliveries in one document (Figure 5).

Unlike PostgreSQL, where retrieving a complete order involves joining about ten tables, MongoDB can return the embedded information in a single document.

I executed five queries, including an aggregation to calculate revenue per vendor. I also generated 20,000 product documents and tested a product-name index. The query improved from a collection scan examining 20,000 documents to an index scan examining one (Figure 6).

I consider MongoDB suitable for flexible catalogue data and order reads, while PostgreSQL remains better suited to stock and payment operations that require relational constraints and transactions.

6. Portfolio 5: Distributed and Cloud Database Exercise

I configured a MongoDB replica set with three database nodes on one laptop: one primary and two secondary nodes (Figure 7). The primary handled writes and replicated data to both secondary nodes. I confirmed that secondary nodes rejected direct write attempts.

When I stopped the primary, a secondary was elected within seconds and accepted new writes. The old primary later rejoined as a secondary and caught up with the changes.

Write concern tests showed the effect of node availability. w: "majority" was acknowledged with one node down, while w: 3 timed out because all three nodes were unavailable (Figure 8).

I also proposed sharding orders by customer ID, using strong consistency for orders and payments, and eventual consistency for catalogue reads. Cloud deployment across multiple availability zones was also discussed. These were design proposals, not implemented features. All three nodes shared one physical machine, limiting the resilience demonstrated.

7. Individual Technical Reflection

One of the key decisions I made in designing Shoptackles was separating products from vendor offers. This allowed different vendors to sell the same product at different prices and maintain their own stock records. During the transaction exercises, I also learned how easily concurrent operations can affect

stock accuracy. In particular, testing two sessions attempting to purchase the same available stock helped me understand why database locking is important and how SELECT ... FOR UPDATE prevents both transactions from processing the same stock simultaneously.

The query optimisation exercise also changed how I approach database performance. I initially expected indexes to improve every query significantly, but the execution plans showed that the results depend on the query and the amount of data being accessed.

8. References and AI Use Declaration

References

- PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*. https://www.postgresql.org/docs/17/
- MongoDB Inc. *MongoDB Manual*. https://www.mongodb.com/docs/manual/
- dbdiagram.io. *Database Diagram Tool*. https://dbdiagram.io/

AI Use Declaration

I used Claude (Anthropic) to explain database concepts, review my design decisions, and assist with drafting SQL and JavaScript scripts and organising the documentation.

I executed every script myself in PostgreSQL and MongoDB, captured the evidence from my own databases, compared the results with the expected behaviour and reviewed the documentation to ensure I can explain each part of the work.

Supporting evidence

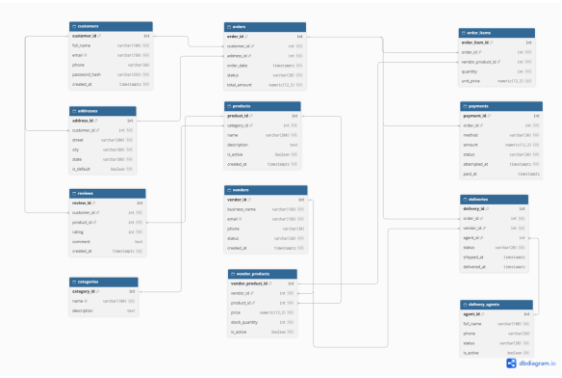


Figure 1: ER diagram of the 12 tables (Portfolio 1)

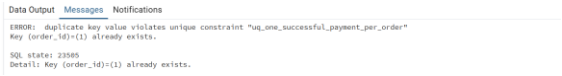


Figure 2: Second successful payment rejected (Portfolio 1)

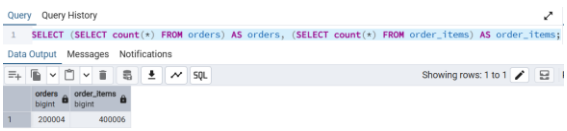


Figure 3: Counts unchanged after rollback (Portfolio 3)

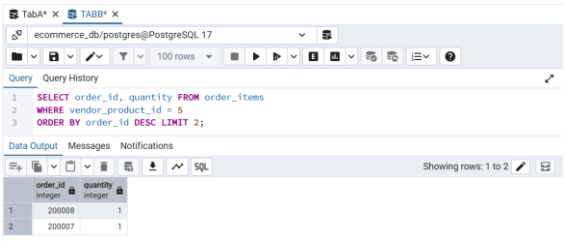


Figure 4: Lost update: two orders, one unit (Portfolio 3)

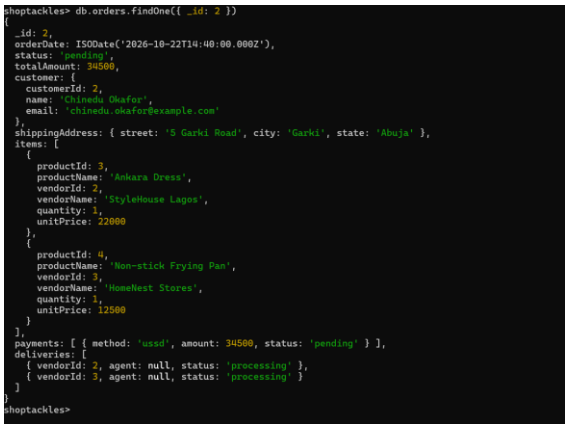


Figure 5: A whole order in one document (Portfolio 4)



Figure 6: Index scan, 1 document examined (Portfolio 4)

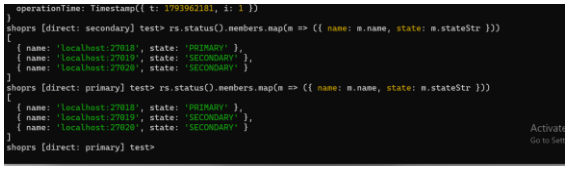


Figure 7: One primary, two secondaries (Portfolio 5)



Figure 8: w: 3 times out with a node down (Portfolio 5)